

iOSDC Japan 2026 ・ Regular Talk ・ 40 min

映像変換サーバーなしで iPhone端末内で HLSを生成してライブ配信

Hikaru Sato @SatoHikaruDev

playlist.m3u8

init.mp4

000001.m4s

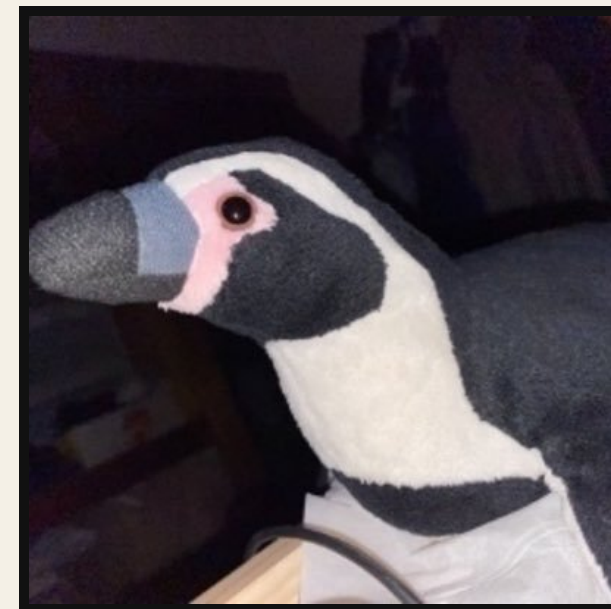
— ABOUT ME

Hikaru Sato

@SatoHikaruDev

主に iOS / Android / Ruby on Rails のアプリ開発をしています

MomentNow という「今この瞬間」の動画をHLSで配信し、
URLで共有できる iOSアプリ を個人開発してます



今回は MomentNow での端末内HLS生成の仕組みや実装を分解します

話すこと

- 01 HLSがライブになる仕組み
- 02 端末内でHLSを生成
- 03 映像と音声の時間軸調整
- 04 約2秒のfMP4を生成
- 05 playlistの更新
- 06 配信の完了

01

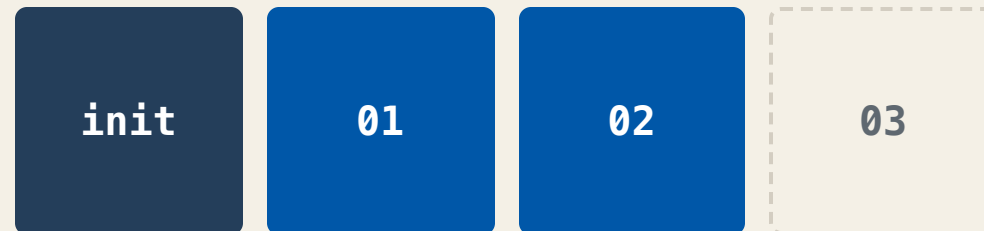
HLSで「ライブ」になる仕組み

短い動画ファイル + 更新され続けるplaylist

— HLS LIVE IN ONE SENTENCE

HLSは、短い動画と 更新されるplaylistをHTTPで配る

```
#EXTM3U
#EXT-X-MAP: init.mp4
#EXTINF: 2.000
seg/000001.m4s
#EXTINF: 2.000
seg/000002.m4s
```



EXT-X-MAP

最初に読む設定ファイル

EXTINF

次のsegmentの長さ

URI

次に取得するファイル

— THREE FILE TYPES

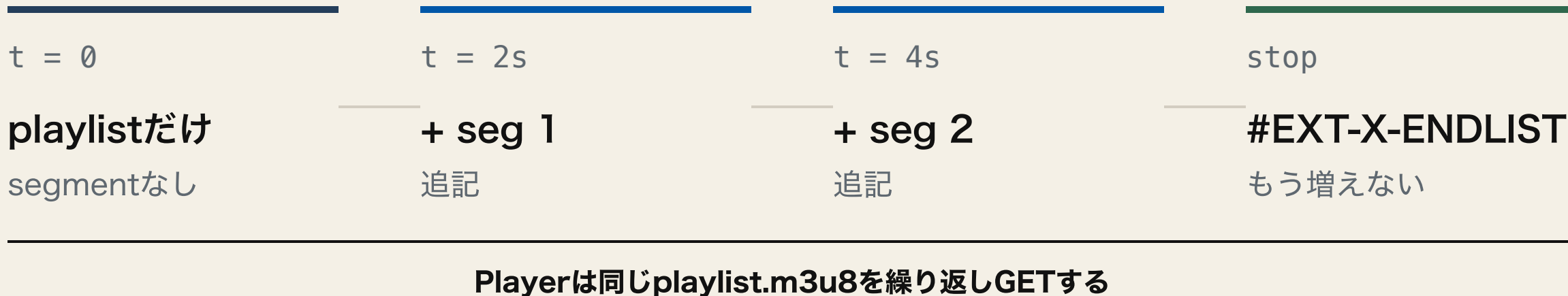
HLSライブ配信を構成する、3種類のファイル

M3U8	MP4	M4S
playlist.m3u8 更新される再生順	init.mp4 再生を始めるための設定	000001.m4s 2秒前後の映像と音声

init.mp4（再生準備） + m4s（約2秒） + playlist（順番） = 最初の再生

— WHY IT IS LIVE

playlistが増えるたび、 Playerは次のsegmentを取得する



— FROM HLS TO THIS TALK

一般的には、サーバーが受信した映像をHLSへ変換する



今回なくすのは、映像を受信し続けて変換するサーバー

1ファイルだと撮影後、共有するためのアップロードに時間がかかる

撮影中からアップロードできていれば共有も楽

撮影終了後

recording.mov

大きな1ファイルを送信



撮影中

init.mp4

001.m4s

002.m4s

小さく区切って順次送信

macOS向けAppleサンプルが、iPhone実装の突破口になった

2025.03

AIで最初の試作

再生できるHLSとして
実用まで至らず

Apple fmp4Writer

正しい生成手順を確認

Writer設定・時刻補正
segment delegate

iPhone local

fMP4生成に成功

Camera / Micから
init.mp4 + .m4s

MomentNow

ライブ配信へ発展

撮影中からupload
playlistを更新

iPhoneで生成できたことが、個人アプリのライブ配信実装の起点

— THIS TALK'S IMPLEMENTATION

iPhoneが生成したHLSを 撮影中からS3へ公開する

映像と音声を受け取る

Camera + Mic



約2秒に分ける

AVAssetWriter



S3へ保存する

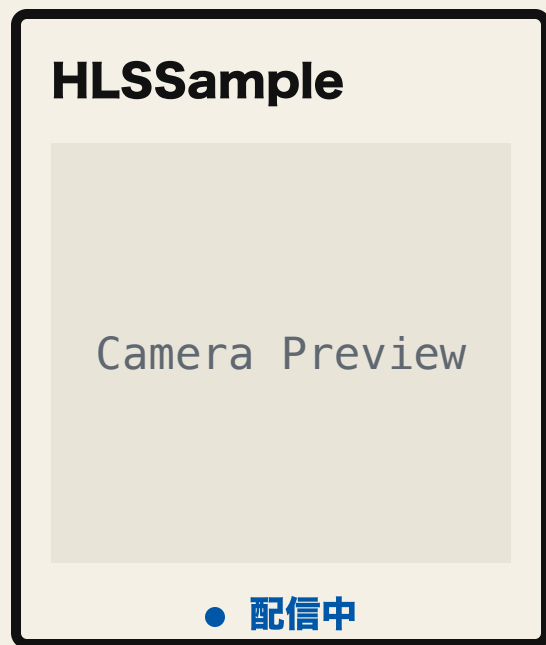
presigned PUT
URL



再生順を公開する

commit API

仕組みが見えたところで、公開サンプルアプリを動かす



HTTP
PUT

start	+2s	after PUT
PUT init.mp4	PUT seg 1	PUT playlist

Mac HTTP Server / Viewer

LIVE LOCAL

playlist.m3u8を追従再生

github.com/HikaruSato/iosdc2026HLSSample

Server は「PUTされたファイルを残すだけ」

Mac

```
$ python3 server/server.py
```

Viewer: <http://localhost:8080>

iPhone server URL: <http://192.168.x.x:8080>

PUT init.mp4 / m4s / m3u8

DISK server/data/streams/...

GET 同じファイルをViewerへ返す

「配信開始」で、iPhoneがHLS生成を始める

● 配信開始



Camera + Mic

video / audio

CMSampleBuffer



AVAssetWriter

init.mp4 + m4s

elapsed 0.0 → segment 0 →

最初に届くのは、1回だけのinit.mp4

$t \approx 0$

initialization callback



init.mp4

再生を始めるための設定



PUT /streams/{id}/init.mp4

media segmentより先に、再生の前提を保存する

以後は約2秒ごとに、2つのPUTが続く

segment 1

PUT 000001.m4s

→

公開

PUT playlist.m3u8

→

segment 2

PUT 000002.m4s

→

公開

PUT playlist.m3u8

playlistが指すのは、保存に成功したsegmentだけ

Viewerは最新streamを見つけ、playlistを追いかける

1 秒ごと

GET /api/streams



最新を選択

playlistURL



LIVE

Safari / AVPlayer / hls.js

STREAM **stream-...** SEGMENTS **1 → 2 → 3** PLAYLIST /streams/.../playlist.m3u8

配信後に残る、3種類のHLSファイル

```
server/data/streams/stream-20260818-225315-C7A2AB11/
```

```
├─ init.mp4                                1,136 bytes
├─ playlist.m3u8                          682 bytes
└─ seg/                                   18 files
    000001.m4s 000002.m4s ... 000018.m4s
```

保存先で `ffplay playlist.m3u8` → HLS一式をそのまま再生確認

CameraからViewerまで、4段階で考える

01 取り込む

Camera / Mic →
CMSampleBuffer

→

02 分割する

約2秒分 → init.mp4
/ .m4s

→

03 保存する

ファイル本体 → Mac
/ S3

→

04 公開する

playlistへ追加 →
Viewer

今回の難所は、各境界で「順序」を失わないこと

— ONE FRAME'S JOURNEY

1 frameは、約2秒のsegmentへ まとめてから届ける



frame単体は送らない。約2秒分を保存してから、playlistで見えるようにする

— SAME SHAPE, DIFFERENT DESTINATION

同じHLSを、MacまたはS3へ置く

公開サンプル

iPhone

HTTP PUT

Mac filesystem

HTTP GET

Local Viewer

MomentNow

iPhone

presigned
PUT

Amazon S3

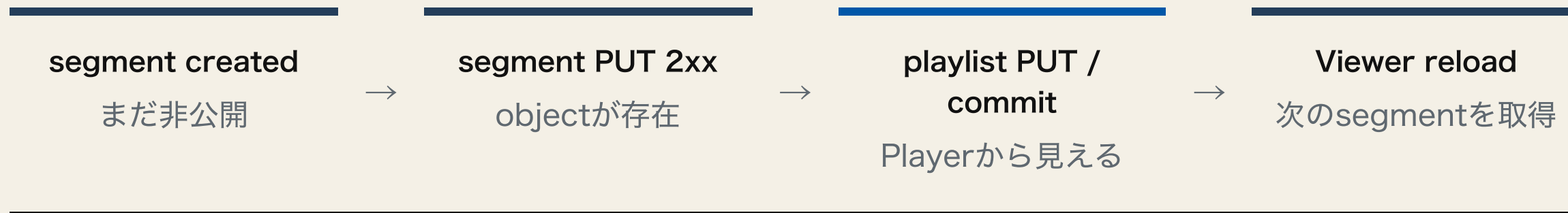
CloudFront

Web Viewer

init.mp4とm4sのバイト列は、どちらもiPhoneが生成する

— CONTROL PLANE

公開タイミングは、playlistが制御する



保存と公開を分ける・playlistはno-cache、init / m4sはlong-cache

fMP4は「設定」と「再生可能な断片」を分ける

1 配信に1つ

init.mp4

Playerが再生を始めるための
設定

+

約2秒ごと

000001.m4s

実際の映像と音声

=

playlistが結ぶ

再生可能

Playerはこの組み合わせを取
得

AVAssetWriterはsegmentを作る。playlistは作らない。

AVFoundation

encode + segment

- H.264 / AAC
- init.mp4
- 約2秒のmedia segment

App

順番を付けて公開

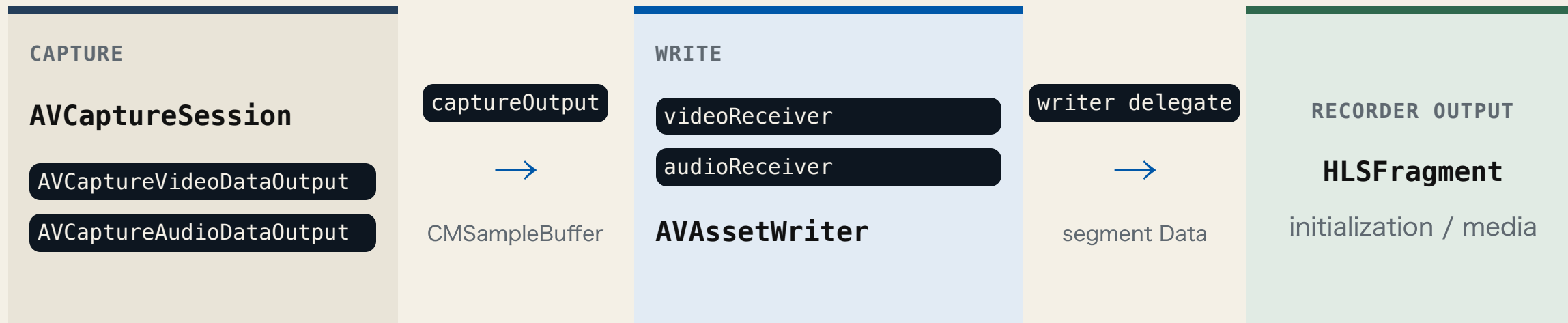
- 再生順番号 (seq)
- S3 upload
- 公開可能なseqを通知

02

Camera / Micから、 fMP4 Dataを取り出す

AVCaptureSession → DataOutput → SampleBufferReceiver → AVAssetWriterDelegate

6つのSDK objectを、2つのdelegateで接続



左から右へ、撮影データを「HLSとして保存できるData」へ変える

— CAPTURE TOPOLOGY

CaptureSessionが、Camera / Micを DataOutputへつなぐ

Back camera

AVCaptureDeviceInput

Microphone

AVCaptureDeviceInput

AVCaptureSession

VideoDataOutput

video CMSampleBuffer

AudioDataOutput

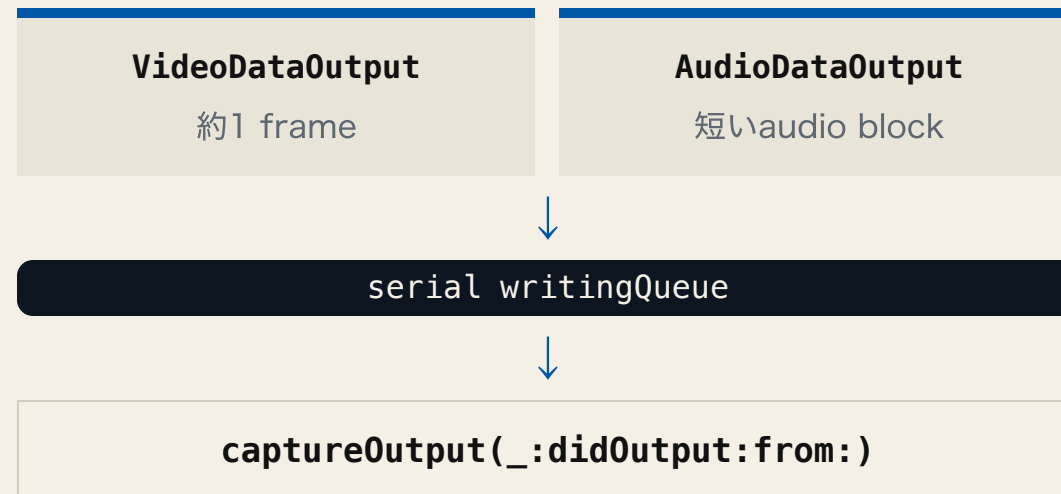
audio CMSampleBuffer

完成した録画ファイルではなく、撮影中のframe / audio blockを受け取る

— DATA OUTPUT DELEGATE

delegateを設定すると、CMSampleBufferが届き始める

```
1 session.addOutput(videoOutput)
2 session.addOutput(audioOutput)
3
4 videoOutput.setSampleBufferDelegate(
5     self, queue: writingQueue
6 )
7 audioOutput.setSampleBufferDelegate(
8     self, queue: writingQueue
9 )
```



VideoとAudioを同じserial queueで受け取り、Writerの状態を1か所で更新する

— URL-LESS HLS WRITER

HLS用Writer

```
1 let writer = AVAssetWriter(  
2     contentType: .mpeg4Movie  
3 )  
4 writer.outputFileTypeProfile = .mpeg4AppleHLS  
5 writer.preferredOutputSegmentInterval = .init(  
6     seconds: 2, preferredTimescale: 600  
7 )  
8 writer.initialSegmentStartTime = startTimeOffset  
9 writer.delegate = self
```

AVAssetWriter

HLS
profile ·
URLなし

→
delegate
→

initialization
Data

separable Data

Writer自身はファイルへ保存せず、生成したfMP4をDataで返す

— INPUT → RECEIVER → WRITER

Receiverが、sampleの書き込み口になる

```
1 let videoInput = AVAssetWriterInput(  
2     mediaType: .video,  
3     outputSettings: videoSettings()  
4 )  
5 let audioInput = AVAssetWriterInput(  
6     mediaType: .audio,  
7     outputSettings: audioSettings()  
8 )  
9  
10 self.videoReceiver = writer.inputReceiver(  
11     for: videoInput  
12 )  
13 self.audioReceiver = writer.inputReceiver(  
14     for: audioInput  
15 )
```

video
CMSampleBuffer

videoReceiver

H.264
Input

audio
CMSampleBuffer

audioReceiver

AAC Input

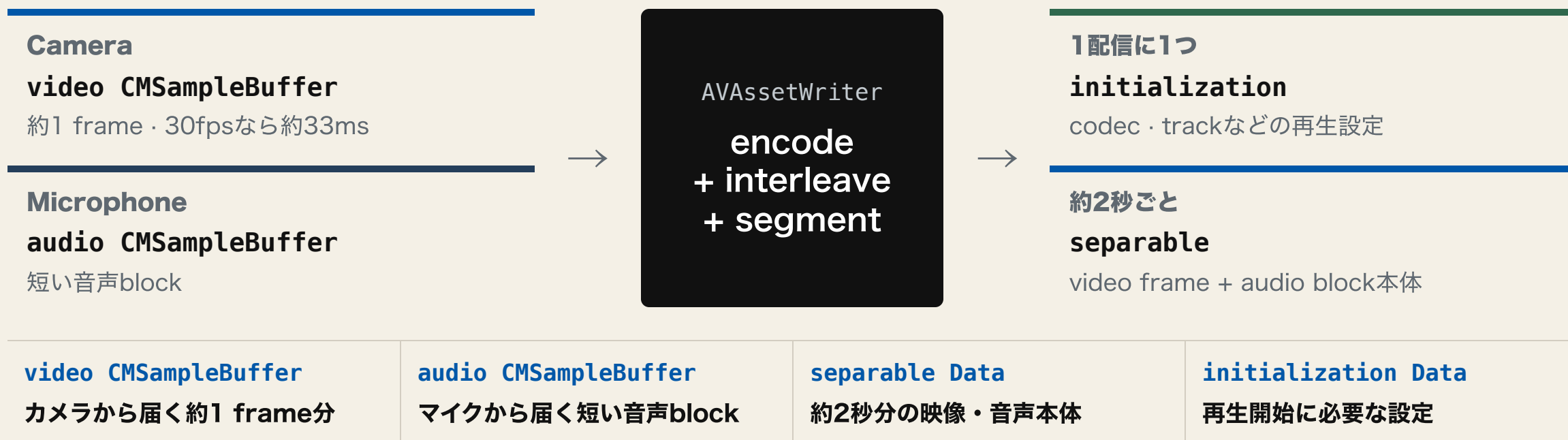
↓ attached to ↓

AVAssetWriter

Inputがencode設定を持ち、ReceiverがCMSampleBufferの入口になる

— CMSAMPLEBUFFER ≠ SEGMENT

多数のCMSampleBufferを、約2秒のsegmentへまとめる



1回のcallbackで、1つのsampleをReceiverへ渡す

DataOutput delegate
captureOutput



Chapter 03
時刻を補正



CoreMedia
CMReadySampleBuffer



SampleBufferReceiver
appendImmediately

```
1 let readySampleBuffer = CMReadySampleBuffer(  
2     unsafeBuffer: transferableSampleBuffer  
3 )  
4 let didAppend = try receiver.appendImmediately(readySampleBuffer)  
5 guard didAppend else { return }
```

true Writerへ渡せた

false 待たずにdrop

throw streamをerror終了

WriterのDataを、Publisherが保存する

.initialization

→ HLSFragment.initialization

→ `init.mp4`

.separable

→ HLSFragment.media(seq, ...)

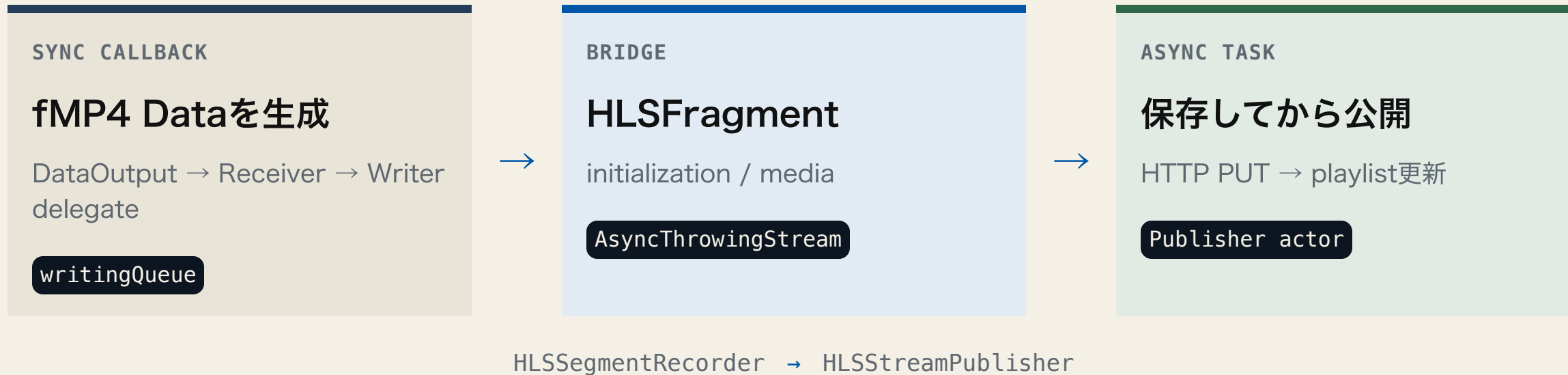
→ `seg/000001.m4s`

```
1  switch segmentType {
2  case .initialization:
3      continuation.yield(.initialization(segmentData))
4  case .separable:
5      segmentIndex += 1
6      continuation.yield(.media(
7          sequence: segmentIndex,
8          data: segmentData,
9          duration: duration
10     ))
11 }
```

Recorderの出力はData。HTTPHLSClientが保存先のpathを組み立てる

— SYNC CALLBACK → ASYNC TASK

生成したDataだけを、公開Taskへ渡す



callbackはDataを渡して戻る。HTTPの完了はPublisher Taskが待つ

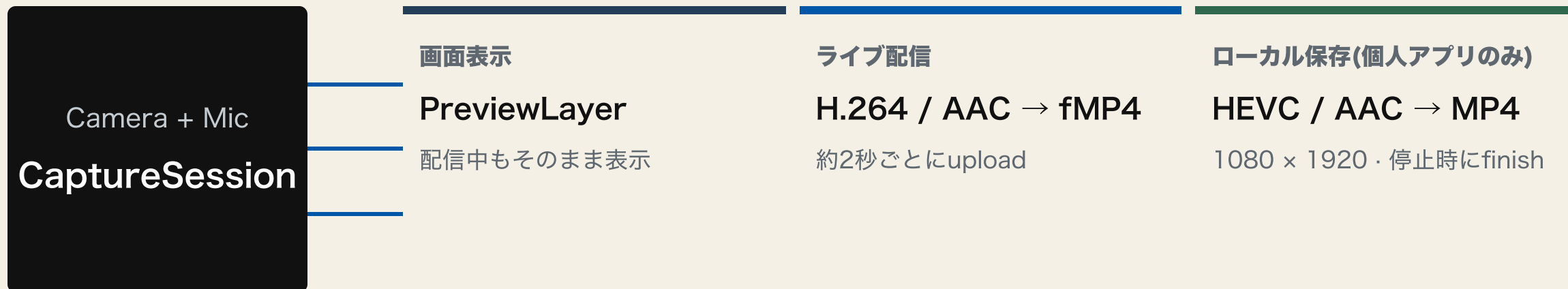
03

映像と音声を、 同じ時間軸でWriterへ渡す

Camera / Mic → CMSampleBuffer → one shared timeline

— ONE CAPTURE · THREE OUTPUTS

1つのCaptureSessionから、3つの用途へ分ける



recording.mp4 → 元動画としてupload + 設定時は写真ライブラリへ保存

公開サンプルはPreview + HLS。MomentNowは保存用Writerも並行する

Cameraの未圧縮映像を AVAssetWriterがH.264に圧縮

Capture

未圧縮映像

video frame

append

Writer

H.264 High

encoded media

```
1 videoOutput.videoSettings = [  
2     kCVPixelBufferPixelFormatTypeKey as String:  
3     kCVPixelFormatType_420YpCbCr8BiPlanarFullRange  
4 ]
```

サンプルは1.5 Mbps ・ 個人アプリは上り回線に合わせて0.9-2.5 Mbps

Writerへ届く前に、2つのguardで落とす

```
captureOutput(_:didOutput:from:)
```

```
isWriting  
== true
```

録画中だけ

```
CMSampleBufferDataIsReady
```

利用可能なbufferだけ

```
startWriterIfNeeded  
→ append
```

```
1 guard isWriting else { return }  
2 guard CMSampleBufferDataIsReady(sampleBuffer) else { return }
```

— SESSION START

最初のvideo frameで、全trackを開始

audio CMSampleBuffer

まだappendしない

first video frame

writer.start()

startSession(at: 10s)

video + audio共通

```
1 guard output === videoOutput else { return }
2 guard !didStartSession, let writer else { return }
```

— PTS = CMSAMPLEBUFFER TIMESTAMP

CMSampleBufferの撮影時刻を、Writerの開始時刻へ移す

58342.31は何秒目？ 配信開始を0とした経過秒ではなく、CaptureSessionの同期時計が示す位置

Capture PTS

CaptureSession

の同期時計

58342.31

– firstVideoPTS + 10s

Writer

timeline

配信内の時刻

10.00

10秒起点は今回のWriter設定。HLS仕様の固定値ではない

— ONE DELTA

時刻補正は、全CMSampleBufferへの平行移動

delta = 10s - firstVideoPTS

adjustedPTS = sourcePTS + delta

— A/V SYNC

VideoとAudioを別々に補正しない

video	58342.31	<u>same delta</u>	video	10.00
audio	58342.29		audio	9.98

相対差を保ったまま、Writerのtime rangeへ移す

04

映像と音声を、 約2秒のfMP4へ分ける

Four HLS settings and segment boundaries

— HLS-SPECIFIC · FOUR KNOBS

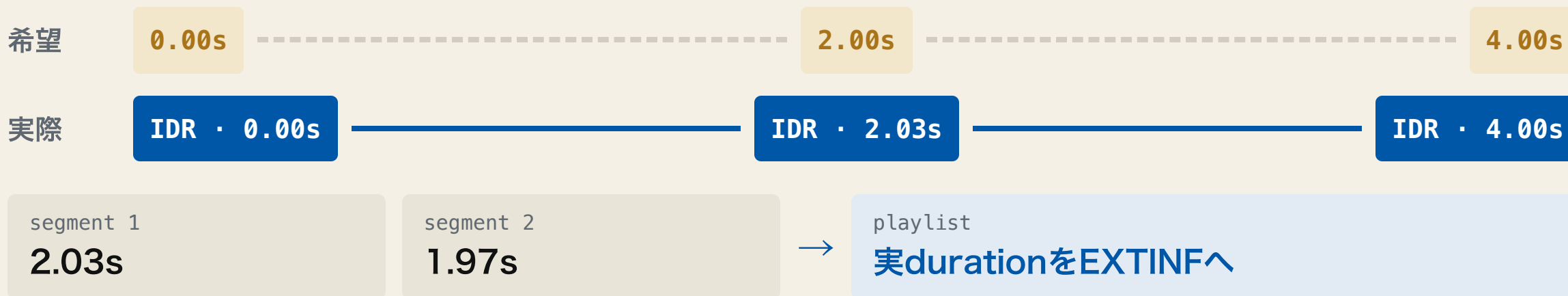
HLS出力は、4つのpropertyで有効になる

- 01 `outputFileTypeProfile` Apple HLS向けfMP4
- 02 `preferredOutputSegmentInterval` 希望するsegment間隔
- 03 `initialSegmentStartTime` 最初のsegment開始時刻
- 04 `delegate` 生成されたDataの受け口

`HLSSegmentRecorder.setupWriterLocked()`

— PREFERRED INTERVAL

2秒ぴったりではなく、2秒付近のIDRで切る



IDR ほかのframeを参照せず、そこから単独で再生を始められるkeyframe

2秒は設定値。playlistにはAVAssetSegmentReportの実durationを書く

— KEYFRAME ALIGNMENT

2秒境界には、2秒以内のIDRが必要

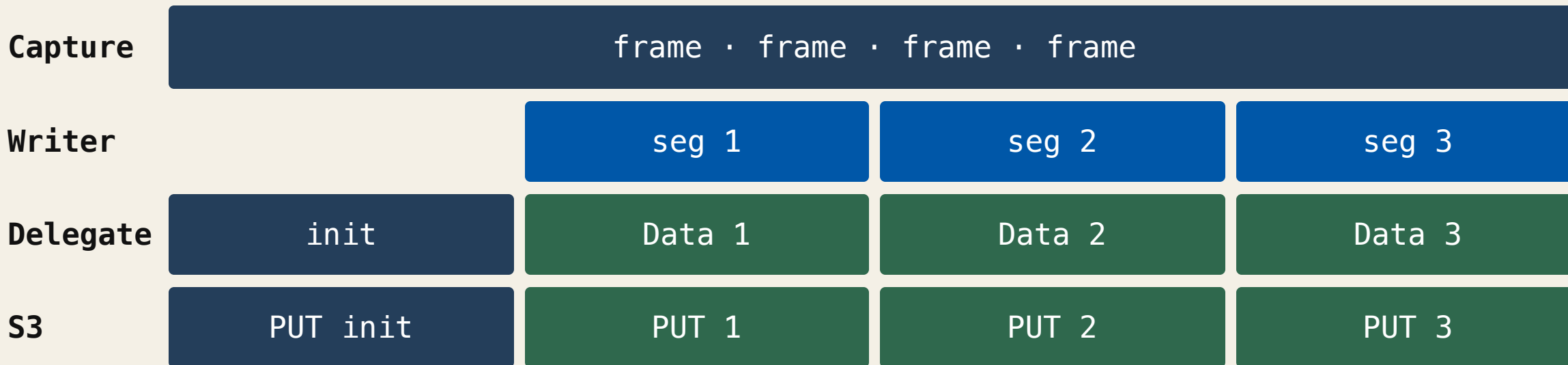


```
1 AVVideoMaxKeyFrameIntervalDurationKey: config.segmentSeconds
```

IDR ほかのframeを参照せず、そこから再生を始められるkeyframe

— GENERATION TIMELINE

Captureを止めず、約2秒ごとにDataを出す



Writerのfinishを待たず、生成とS3 uploadを並行できる

— BACKPRESSURE DECISION

Writerが詰まったら、古いframeを捨てる

Camera	frame 1	frame 2	frame 3	frame 4
Writer	append	append	busy	drop

待って貯める

古い映像が残り、遅延が増える

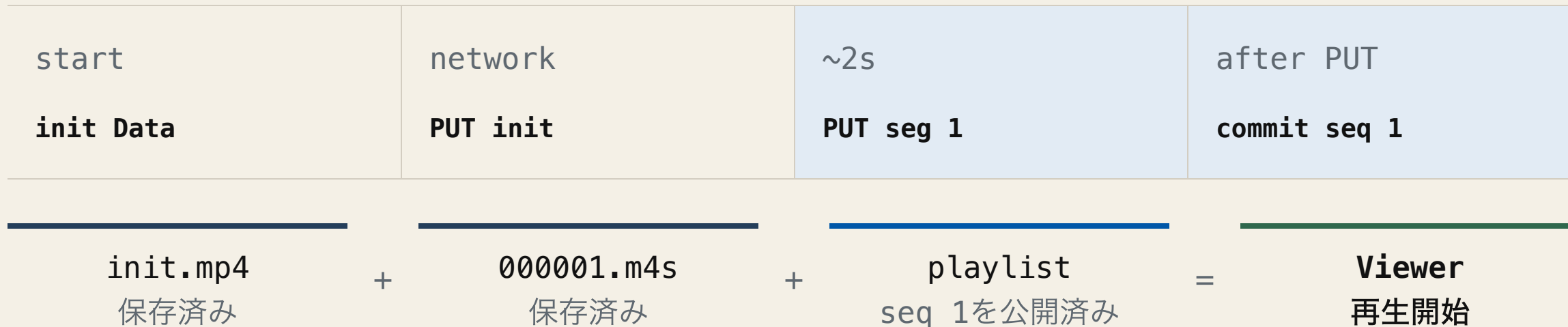
待たずに落とす

少しカクつくが、現在へ追いつく

`appendImmediately == false` は意図的なdrop

— VISIBLE OUTPUT

再生開始は、init PUTと最初のcommitの後



playlist公開後に、状態を確定する

```
1 var nextManifest = manifest
2 nextManifest.addSegment(seq: seq, durationSec: durationSec)
3
4 try await client.putPlaylist(streamId: streamId, text: nextManifest.text)
5 manifest = nextManifest
```

current
segments 1...N

copy

candidate
+ segment N+1

PUT
2xx

local stateを確定
current =
candidate

後半のcommit APIとは別。ここではiPhone自身がplaylist本文を書く

— DRAIN BEFORE FINISH

stopは、3つの非同期処理を順番に閉じる

1 Recorder

capture停止 → finishWriting

最後のfragmentが出る

→

2 Stream

continuation.finish()

fragment列を閉じる

→

3 Publisher

uploadTask.value

ENDLIST PUTまで待つ

「撮影停止」と「配信終了」は、同じ瞬間ではない

05

保存できたsegmentだけを、 playlistへ公開する

Save bytes first, then make the segment visible to viewers

「保存先を受け取る → 保存する → 公開する」に分ける

APPの操作	MomentNow	結果
S3への署名付きURLを受け取る	POST /presign	presigned PUT URL
segment本体を保存する	PUT → S3	objectが存在する
playlistへ載せる	POST /commit	Viewerから見える
playlistを取得する	GET → CloudFront	再生が進む

S3は保存先の選択。HLSで重要なのは「保存してから公開」

— INITIALIZATION SEGMENT

init.mp4は、最初に1度だけPUT

```
1 guard !didUploadInit else { return }
2 let ticket = try await api.presign(
3     streamId: stream.streamId, kind: "init", seq: nil
4 )
5 try await api.put(
6     to: URL(string: ticket.putUrl)!, data: initData,
7     contentType: "video/mp4"
8 )
9 didUploadInit = true
```

— MEDIA SEGMENT

mediaごとにpresign → PUT → commit

```
1 let ticket = try await api.presign(  
2   streamId: stream.streamId, kind: HLSUploadKind.segment,  
3   seq: seq  
4 )  
5 try await api.put(  
6   to: URL(string: ticket.putUrl)!, data: segmentData,  
7   contentType: HLSContentType.segmentM4S  
8 )  
9 _ = try await api.commit(  
10  streamId: stream.streamId, seq: seq, durationSec: dur,  
11  isLast: false, targetDurationSec: targetDurationSec  
12 )
```

— S3 PRESIGNED PUT URL

サーバーが、S3へPUTするための署名付きURLを発行する

DO NOT

AWS credentialを内包

漏えい範囲が広い



PRESIGNED URL

S3へ1 objectだけPUT

object key / 有効期限を署名

iPhoneは発行されたURLへDataをPUTするだけ。AWS credentialは持たない

```
1 var request = URLRequest(url: presignedURL)
2 request.httpMethod = "PUT"
3 request.setValue(contentType, forHTTPHeaderField: "Content-Type")
4 request.httpBody = data
```

— PUBLISH AFTER BYTES

PUT成功より先に、playlistへseqを出さない

BAD

commit

→

Viewer GET

→

404

GOOD

S3 PUT完了

→

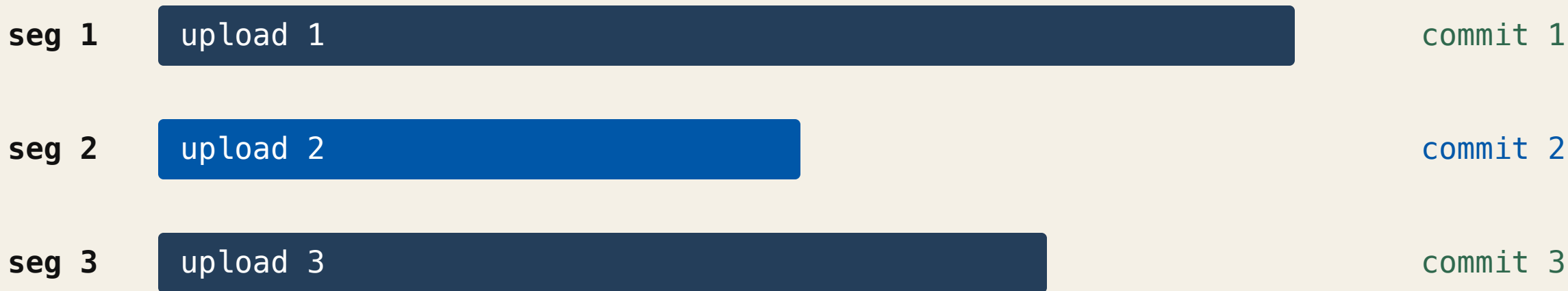
commit

→

playlist更新

— OUT-OF-ORDER RACE

現在のcommit APIはseq順へ並べ直すが、 欠番は待たない



completion **2, 1, 3** $\neq$ playback **1, 2, 3**

理想は、連続したseqまでだけを公開する「公開済み境界」を持つこと

06

Writerを待たせず、 最後の公開完了まで管理する

Callback → Task → upload completion

callbackでは待たず、TaskへDataを引き渡す

LiveSegmentRecorder

init Data / media Data

WRITING QUEUE

callbackを返す

次のCMSampleBuffer処理へ

SWIFT TASK

Uploaderをawait

presign / PUT / commit

initialization →
uploadInitIfNeeded

separable → uploadSegment segmentReport → duration改善余地

— PUBLIC DEMO → PRODUCTION

個人アプリでは、HLSの外側に4つの制御を足す

AUTH

user / group
ownership

PRESIGN

object単位の短期PUT
権限

COMMIT

公開順とplaylistの同時
更新を制御

DELIVERY

CloudFront / ticket /
status

AVAssetWriterの後ろを差し替えれば、同じiOS pipelineを使える

— FIT, NOT UNIVERSAL

この構成が効くのは、用途を絞ったとき

GOOD FIT

少人数・短時間・単一品質

新しいiPhone / 安定した上り回線

USE A MEDIA PLATFORM

大規模・長時間・ABR・厳しいSLA

発熱、電池、回線変動を吸収したい

「端末でできる」と「端末でやるべき」は別の判断

— TAKEAWAYS

HLSライブ配信は、4つの順序で成立する

01 時刻 ・ Clock

Capture PTSをWriterのtimelineへ移す

02 区切り ・ Boundary

IDRとsegment intervalをそろえる

03 公開 ・ Upload

DataをS3へPUTしてからcommitする

04 終了 ・ Finish

全送信とfinal commitの終了を待つ

ありがとうございました

APPENDIX

時間が余ったとき／Q&A用

設定値・時刻補正・サーバー運用の詳細

— CONFIG

上り回線に合わせて、配信品質を選ぶ

```
1 let profile: (CGSize, Int) = switch quality {  
2 case .high:    (.init(width: 720, height: 1280), 2_500_000)  
3 case .medium:  (.init(width: 720, height: 1280), 1_500_000)  
4 case .low:     (.init(width: 480, height: 854),    900_000)  
5 }
```

NWPath

constrained /
expensive

presign

API latency

probe PUT

upload throughput

2.0 sec

segment target

カメラ構成前に、録音用AudioSessionを有効化

```
1 let audio = AVAudioSession.sharedInstance()
2 try audio.setCategory(
3     .playAndRecord,
4     mode: .videoRecording,
5     options: [.defaultToSpeaker, .allowBluetoothHFP]
6 )
7 try audio.setActive(true)
```

Capture

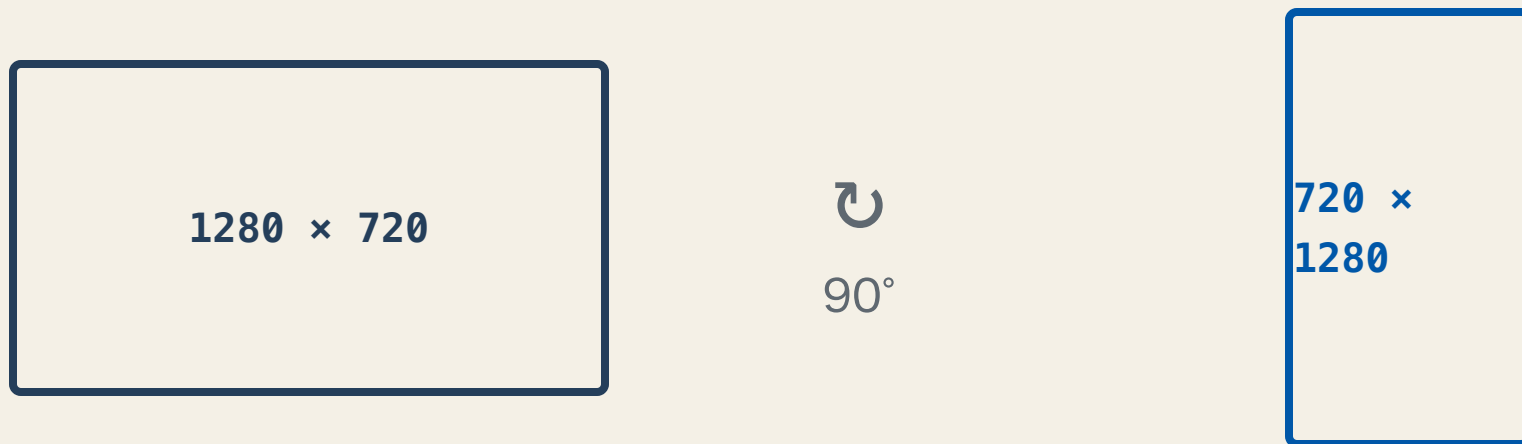
camera + microphone

Route

speaker + Bluetooth HFP

— ORIENTATION

縦向きは、video connectionへ90度を指定



```
1 if let connection = videoOutput.connection(with: .video),  
2   connection.isVideoRotationAngleSupported(90) {  
3   connection.videoRotationAngle = 90  
4 }
```

— COPY TIMING

映像・音声は変えず、時刻情報だけを補正

```
1 let timingInfos = try sampleTimingInfos().map { info in
2     var adjusted = info
3     adjusted.presentationTimeStamp =
4         info.presentationTimeStamp + offset
5     if info.decodeTimeStamp.isValid {
6         adjusted.decodeTimeStamp = info.decodeTimeStamp + offset
7     }
8     return adjusted
9 }
10 let copied = try CMSampleBuffer(copying: self, withNewTiming: timingInfos)
```

PTSと、有効なDTSを同じ量だけ動かす

— STOP IN THE SAME CLOCK

開始と終了を、同じ補正後timelineで閉じる

lastAdjustedPTS

最後にappendした時刻

→

endSession

media rangeを閉じる

→

markAsFinished

video / audio

→

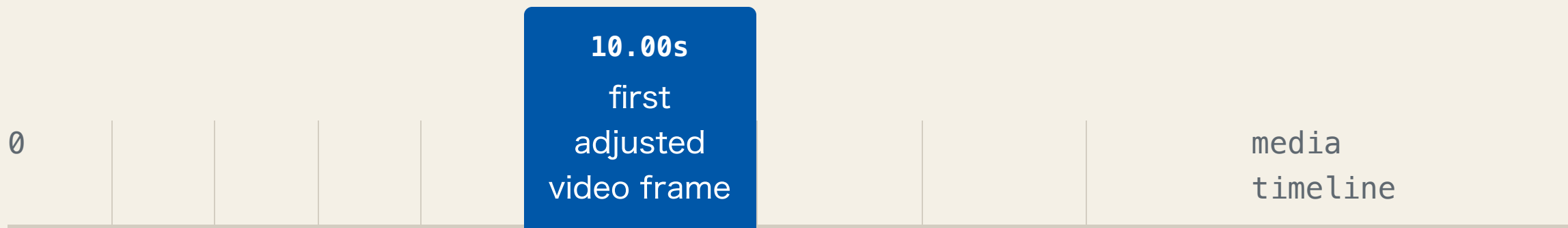
finishWriting

最後のsegmentをflush

```
1 if lastAdjustedPTS.isValid {  
2     writer.endSession(atSourceTime: lastAdjustedPTS)  
3 }
```

— INITIAL START TIME

Writerの最初のsegmentは、10秒起点に固定



```
1 private let startTimeOffset = CMTime(value: 10, timescale: 1)
2 writer.initialSegmentStartTime = startTimeOffset
```

— VIDEO SETTINGS

配信用videoフォーマットは、互換性と上り帯域を優先する

KEY	VALUE	INTENT
AVVideoCodecKey	H.264	広い再生互換性
Width / Height	480p / 720p	quality別を選択
AverageBitRate	0.9–2.5 Mbps	上り回線へ追従
ProfileLevel	High Auto	encoder profile
FrameReordering	false	decode順を単純化

LiveSegmentRecorder.makeHLSVideoSettings()

— AUDIO SETTINGS

AudioはAAC mono、品質別に48-96kbps

FORMAT	CHANNEL	SAMPLE RATE	BITRATE
AAC	mono	44.1 kHz	48-96 kbps

```
1  [  
2    AVFormatIDKey: kAudioFormatMPEG4AAC,  
3    AVNumberOfChannelsKey: 1,  
4    AVSampleRateKey: 44_100,  
5    AVEncoderBitRateKey: config.audioBitrate  
6  ]
```

実測durationを使い、取れなければ2秒

REPORT

video track duration

EXTINFへ実際の長さ。とくに最後のsegmentは2秒にならないので必要

FALLBACK

config.segmentSeconds

無効・未取得なら2.0

```
1 let duration = report?.trackReports
2   .first { $0.mediaType == .video }?
3   .duration.seconds
4
5 guard let duration, duration.isFinite, duration > 0 else {
6   return config.segmentSeconds
7 }
8 return duration
```

サンプルは、同じobjectを 最大3回まで再送する



segment PUTが失敗したら、そのsegmentをplaylistへ載せない

PUT中のファイルを、Viewerへ見せない

1 temporary file

request bodyを書き込む



2 flush + fsync

長さまで書き切る



3 os.replace

完成品へ一気に置換

playlistの途中状態や、半分だけのm4sをGETさせない

cache方針は、playlistとsegmentで分ける

OBJECT	CACHE	HTTP
playlist.m3u8	no-store / no-cache	毎回最新を取得
init.mp4 / m4s	max-age=31536000	Range / 206対応

S3 / CloudFrontでの設定